CineThink

Roadmap de Desarrollo

v1.0 — 2026

Campo	Valor
Fases totales	7 fases (F0 a F6)
Subfases totales	20 subfases
Funcionalidades MVP	12 — distribuidas en F1 a F6
Criterio de avance	Criterios de completitud, no fechas
Stack backend	FastAPI + Supabase + Render.com
Stack frontend	React + TypeScript strict + Vite + Tailwind
Motor de IA	Groq API — Llama 3.3 70B
Datos de peliculas	TMDB API

Como leer este documento

Cada fase cubre un modulo completo del producto. Dentro de cada fase hay subfases que son las unidades minimas de trabajo: una subfase se trabaja completa antes de pasar a la siguiente.

Cada subfase tiene cinco secciones fijas: Objetivo (que resuelve esta subfase), Tareas (que hay que construir exactamente), Listo cuando (criterios de completitud — la subfase no esta terminada hasta que todos se cumplen), Estudiar (lo que necesitas aprender para completarla, especifico y aplicable), y Bloqueos (riesgos conocidos que pueden frenar el avance).

OBJETIVO	Que problema resuelve esta subfase en el producto
TAREAS	Lista exacta de lo que hay que construir o configurar
LISTO CUANDO	Criterios de completitud — todos deben cumplirse para avanzar
ESTUDIAR	Conceptos o documentacion especifica a leer antes o durante

BLOQUEOS	Riesgos conocidos que pueden frenar esta subfase

Vision general del Roadmap

Fase	Nombre	Que construye	Funcionalidades	Resultado
F-0	**Fundamentos**	Monorepo, CI/CD, Supabase, Render, Vercel	Ninguna — infraestructura	Proyecto desplegable en produccion desde el dia 1
F-1	**Autenticacion**	Registro, login, sesion persistente	F-01, F-02	Usuario puede crear cuenta e iniciar sesion
F-2	**Nucleo peliculas**	Busqueda TMDB, lista personal de vistas	F-04, F-05	Usuario puede buscar y registrar peliculas vistas
F-3	**Motor de analisis**	Chat con IA, etiquetas semanticas, historial	F-06, F-07, F-08, F-09, F-10	Usuario puede analizar peliculas con IA
F-4	**Perfil**	Perfil semantico, preferencias, memoria, instrucciones	F-11, F-12, F-13, F-14	Perfil completo y personalizable activo
F-5	**Recomendaciones**	Motor de recomendaciones desde el perfil	F-15	Usuario recibe recomendaciones personalizadas
F-6	**Cierre MVP**	Exportacion, pulido UI, tests e2e, deploy estable	F-16	MVP completo listo para usuarios reales

0 FASE — Fundamentos e Infraestructura

Setup del proyecto — nada se construye antes de que esto este listo

Esta fase no produce funcionalidades visibles para el usuario. Produce la base sobre la que se construye todo lo demas: estructura del proyecto, conexion con todos los servicios externos, y un pipeline de deploy que funciona desde el dia uno.

REGLA	La Fase 0 esta completa cuando una pagina en blanco con 'CineThink' esta deployada en Vercel, el backend responde en Render, y Supabase tiene las tablas base creadas. Si alguno de esos tres no funciona, la fase no esta terminada.

F0-A Estructura del monorepo

	Funcionalidades: Ninguna — infraestructura pura
OBJETIVO	Crear la estructura de carpetas del proyecto, inicializar el repositorio en GitHub, y configurar las herramientas base del frontend y backend.
TAREAS	• Crear repositorio en GitHub: cinethink • Estructura de carpetas: /frontend (Vite + React + TS), /backend (FastAPI + Python), /docs • Inicializar frontend: npm create vite@latest con template react-ts • Instalar dependencias base frontend: tailwindcss, zustand, react-router-dom, axios • Inicializar backend: crear entorno virtual Python, instalar fastapi, uvicorn, supabase-py, python-dotenv • Archivo .gitignore correcto para ambos: node_modules, __pycache__, .env, venv • Archivo README.md base con descripcion del proyecto y estructura • Primer commit con estructura limpia y funcional
LISTO CUANDO	• El frontend corre localmente con npm run dev sin errores • El backend corre localmente con uvicorn main:app --reload sin errores • El repositorio esta en GitHub con el primer commit • El .gitignore excluye correctamente archivos de entorno y dependencias
ESTUDIAR	• Vite: documentacion oficial de setup con React y TypeScript — vitejs.dev/guide • FastAPI: tutorial oficial Getting Started — fastapi.tiangolo.com/tutorial • Estructura de monorepos: como organizar frontend y backend en el mismo repo

F0-B	**Configuracion de Supabase** *Funcionalidades: Ninguna — infraestructura pura*
OBJETIVO	Crear el proyecto en Supabase, definir el schema inicial de la base de datos, y conectar el backend con Supabase.
TAREAS	• Crear proyecto en supabase.com — plan gratuito • Crear las tablas base en el SQL Editor de Supabase: profiles, movies_watched, analysis_sessions, analysis_messages, semantic_tags, user_profile, recommendations • Definir tipos correctos por columna, primary keys, foreign keys, y timestamps automaticos (created_at, updated_at) • Habilitar Row Level Security (RLS) en todas las tablas desde el inicio — cada usuario solo ve sus propios datos • Escribir las politicas RLS basicas: SELECT, INSERT, UPDATE, DELETE solo para el usuario autenticado dueno del registro • Agregar las variables de entorno al backend: SUPABASE_URL y SUPABASE_SERVICE_KEY en archivo .env • Probar la conexion desde el backend con un script simple que liste registros de una tabla
LISTO CUANDO	• Todas las tablas existen en Supabase con el schema correcto

	• RLS esta habilitado en todas las tablas • El backend puede conectarse a Supabase y ejecutar una query sin errores • Las variables de entorno estan en .env y NO en el codigo fuente
ESTUDIAR	• Supabase: seccion Database de la documentacion oficial — supabase.com/docs/guides/database • Row Level Security: concepto y como escribir politicas — supabase.com/docs/guides/auth/row-level-security • supabase-py: SDK oficial de Python — github.com/supabase-community/supabase-py
BLOQUEOS	• *RLS mal configurado puede bloquear todas las queries del backend — leer la documentacion antes de activarlo* • *La SERVICE_KEY del backend bypasea RLS — nunca exponerla al frontend*

F0-C	**Deploy inicial en Vercel y Render** *Funcionalidades: Ninguna — infraestructura pura*
OBJETIVO	Conectar el repositorio de GitHub con Vercel (frontend) y Render (backend) para tener deploy continuo desde el primer commit.
TAREAS	• Conectar el repositorio de GitHub a Vercel — importar proyecto, configurar build command (npm run build) y output directory (dist) • Verificar que el frontend deploya correctamente en Vercel al hacer push a main • Crear un nuevo Web Service en Render conectado al mismo repositorio • Configurar el build command de Render: pip install -r requirements.txt • Configurar el start command de Render: uvicorn main:app --host 0.0.0.0 --port $PORT • Agregar las variables de entorno en el dashboard de Render: SUPABASE_URL, SUPABASE_SERVICE_KEY • Crear endpoint GET /health en FastAPI que retorna {status: ok} — sirve para verificar que el backend esta vivo • Verificar que el backend responde en la URL de Render • Agregar la URL del backend como variable de entorno en Vercel: VITE_API_URL
LISTO CUANDO	• El frontend esta accesible en una URL de Vercel (.vercel.app) • El backend responde en una URL de Render (.onrender.com) con GET /health retornando 200 • Cada push a main en GitHub triggerea un nuevo deploy en ambos servicios automaticamente • Las variables de entorno estan configuradas en ambos servicios — nunca en el codigo
ESTUDIAR	• Vercel: como conectar un repositorio de GitHub y configurar el build — vercel.com/docs • Render: como crear un Web Service con Python — render.com/docs/web-services • Variables de entorno en Vite: diferencia entre VITE_ prefix y variables del servidor

BLOQUEOS	
	• *Render en plan gratuito hiberna los servicios inactivos — el primer request despues de inactividad tarda ~30s. Es aceptable para MVP.* • *Vercel puede fallar el build si hay errores de TypeScript — asegurarse de que el proyecto compila sin errores antes del primer deploy*

1 FASE	**Autenticacion** *El usuario puede crear una cuenta, iniciar sesion y mantenerla activa*

Al finalizar esta fase, un usuario real puede registrarse en CineThink, iniciar sesion, y navegar la app autenticado sin tener que volver a loguearse en cada visita. Es la base de todo lo que viene despues.

F1-A	**Registro de usuario** *Funcionalidades: F-01*
OBJETIVO	Construir el formulario de registro en el frontend y el endpoint de creacion de usuario en el backend usando Supabase Auth.
TAREAS	• Endpoint POST /auth/register en FastAPI: recibe email, password, username. Llama a supabase.auth.sign_up(). Crea registro en tabla profiles con el user_id retornado. • Validaciones en el backend: email con formato valido, password minimo 8 caracteres, username no vacio • Pagina /register en el frontend con formulario: campos email, contrasena, nombre de usuario • Validacion en el frontend antes de enviar: campos no vacios, formato de email, contrasena minimo 8 chars • Llamada al endpoint POST /auth/register desde el frontend con axios • En exito: guardar el token de sesion retornado por Supabase y redirigir al dashboard • Manejo de errores visibles al usuario: email ya registrado, contrasena debil, error de red • Tests: al menos un test de integracion del endpoint en FastAPI (email valido, email duplicado, password corto)
LISTO CUANDO	• Un usuario nuevo puede registrarse con email y contrasena desde la UI • El registro falla con mensaje claro si el email ya existe • El registro falla con mensaje claro si la contrasena tiene menos de 8 caracteres • El registro exitoso crea un registro en la tabla profiles de Supabase • Los tests del endpoint pasan
ESTUDIAR	• Supabase Auth: como funciona sign_up y que retorna — supabase.com/docs/guides/auth • FastAPI: como manejar errores HTTP con HTTPException • React: manejo de formularios con estado local (useState) sin librerias externas para empezar

BLOQUEOS	• *Supabase por defecto envia un email de confirmacion — en desarrollo esto puede bloquear el flujo. Deshabilitar la confirmacion de email en el dashboard de Supabase para el MVP.*

F1-B	**Login y sesion persistente** *Funcionalidades: F-02*
OBJETIVO	Construir el flujo de login y asegurarse de que la sesion persiste entre visitas. El usuario no debe autenticarse cada vez que abre la app.
TAREAS	• Endpoint POST /auth/login en FastAPI: recibe email y password. Llama a supabase.auth.sign_in_with_password(). Retorna el access_token y refresh_token. • Pagina /login en el frontend con formulario: email y contrasena • En login exitoso: guardar access_token en memoria (Zustand store) y refresh_token en localStorage • Implementar logica de persistencia: al cargar la app, verificar si hay un refresh_token en localStorage. Si existe, llamar al endpoint de refresh para obtener un nuevo access_token sin que el usuario tenga que loguearse. • Endpoint POST /auth/refresh en FastAPI: recibe refresh_token, retorna nuevo access_token • Ruta protegida: un Higher Order Component o hook useAuth que redirige a /login si no hay sesion activa • Boton de logout disponible en el header: limpia el store de Zustand y el localStorage, redirige a /login • Manejo de errores: credenciales incorrectas, cuenta inexistente, error de red • Tests: login exitoso, login con credenciales incorrectas, refresh de token
LISTO CUANDO	• El usuario puede loguearse con sus credenciales desde la UI • Al recargar la pagina, el usuario sigue autenticado si ya habia iniciado sesion • El logout limpia completamente la sesion • Las rutas protegidas redirigen a /login si no hay sesion • Los tests del flujo de auth pasan
ESTUDIAR	• JWT: que es un access_token y un refresh_token, por que se usan los dos — jwt.io/introduction • Zustand: como crear un store de autenticacion con persistencia — docs.pmnd.rs/zustand • React Router: como implementar rutas protegidas con Navigate
BLOQUEOS	• *El access_token de Supabase expira en 1 hora por defecto. El flujo de refresh debe funcionar correctamente o el usuario sera deslogueado inesperadamente.*

2 FASE — Nucleo de Peliculas

El usuario puede buscar peliculas reales y registrar las que ha visto

Al finalizar esta fase, el usuario puede buscar cualquier pelicula usando datos reales de TMDB y registrarla en su lista personal. Esta es la primera funcionalidad completamente visible y usable del producto.

F2-A	**Integracion con TMDB API** *Funcionalidades: F-04*
OBJETIVO	Implementar la busqueda de peliculas en tiempo real usando la API publica de TMDB. La key de TMDB vive en el frontend — no requiere proxy.
TAREAS	• Crear cuenta gratuita en themoviedb.org y obtener API key • Agregar VITE_TMDB_API_KEY como variable de entorno en el frontend (y en Vercel) • Crear servicio tmdb.ts en el frontend: funciones searchMovies(query), getMovieDetail(id) • searchMovies: llama a GET https://api.themoviedb.org/3/search/movie?query={q}&api_key={key}. Retorna lista de resultados con id, title, poster_path, release_date, genre_ids, overview. • getMovieDetail: llama a GET https://api.themoviedb.org/3/movie/{id}?api_key={key}&append_to_response=credits. Retorna detalle completo con reparto y director. • Componente SearchBar con debounce de 300ms — no llama a la API en cada keystroke • Componente MovieCard: muestra poster (con fallback si no hay imagen), titulo, ano, genero • Pagina de detalle /movie/:id: sinopsis completa, poster grande, reparto top 5, director, duracion, calificacion TMDB • Estados de carga (skeleton o spinner), error (mensaje claro), y sin resultados
LISTO CUANDO	• La busqueda retorna resultados reales de TMDB en menos de 500ms • El debounce funciona — no se hacen llamadas multiples mientras el usuario escribe • La pagina de detalle muestra todos los campos definidos • Los estados de carga, error y sin resultados se muestran correctamente • La API key no esta visible en el codigo fuente — viene de variable de entorno
ESTUDIAR	• TMDB API: documentacion de los endpoints de busqueda y detalle — developers.themoviedb.org • Debounce: como implementar debounce en React con useEffect o con una libreria como lodash • Como construir la URL del poster de TMDB: https://image.tmdb.org/t/p/w500/{poster_path}

F2-B	**Lista personal de peliculas vistas** *Funcionalidades: F-05*
OBJETIVO	El usuario puede marcar una pelicula como vista, agregar una nota inicial opcional, y ver su lista personal de peliculas vistas.

TAREAS	• Endpoint POST /movies/watched en FastAPI: recibe tmdb_id, title, poster_url, initial_note (opcional). Inserta en tabla movies_watched con el user_id del token JWT. • Middleware de autenticacion en FastAPI: un dependency que extrae el user_id del Bearer token en cada request protegido • Endpoint GET /movies/watched en FastAPI: retorna la lista de peliculas vistas del usuario autenticado, ordenadas por fecha de registro descendente • Endpoint DELETE /movies/watched/:id en FastAPI: elimina una pelicula de la lista (con validacion de que pertenece al usuario) • Boton 'Marcar como vista' en la pagina de detalle de cada pelicula — abre un modal con campo de nota inicial opcional • Pagina /library: lista de todas las peliculas vistas con poster, titulo, fecha en que la marco, y si tiene analisis o no • Filtros en /library: por genero, por ano, por si tiene o no analisis • Boton de eliminar en cada pelicula de la lista con confirmacion • Si el usuario ya marco la pelicula como vista, el boton cambia a 'Ya vista' con opcion de ver el analisis
LISTO CUANDO	• El usuario puede marcar una pelicula como vista desde la pagina de detalle • La pelicula aparece inmediatamente en /library despues de marcarla • La nota inicial se guarda correctamente en Supabase • El usuario puede eliminar una pelicula de su lista • Los filtros de /library funcionan correctamente • Un usuario no puede ver ni modificar las peliculas de otro usuario (RLS activo)
ESTUDIAR	• FastAPI: como usar Depends para inyectar dependencias — el patron para el middleware de auth • JWT en FastAPI: como decodificar y validar un Bearer token en cada endpoint protegido • React: como hacer un modal accesible con estado local
BLOQUEOS	• *El middleware de autenticacion es critico — si no esta bien implementado, cualquier usuario puede ver los datos de otro. Probar esto explicitamente.*

3 FASE — Motor de Analisis

El corazon del producto — el usuario conversa con la IA sobre peliculas

Esta es la fase mas compleja del proyecto. Al terminarla, el ciclo central de valor existe: el usuario puede analizar una pelicula con IA, ver el historial de esa conversacion, y el sistema extrae automaticamente etiquetas semanticas que empiezan a construir el perfil.

F3-A	**Integracion con Groq API y prompt base** *Funcionalidades: F-06 parcial*
OBJETIVO	Conectar el backend con Groq API y construir el prompt de sistema base del motor de analisis. Este es el foundation de toda la IA en CineThink.

TAREAS	• Crear cuenta en console.groq.com y obtener API key — plan gratuito • Instalar groq SDK en Python: pip install groq • Agregar GROQ_API_KEY como variable de entorno en el backend (y en Render) • Crear servicio ai_service.py en el backend: funcion generate_response(messages, system_prompt) que llama a Groq con el modelo llama-3.3-70b-versatile • Construir la funcion build_analysis_prompt(movie_data, user_profile, user_instructions, user_memory) que ensambla el prompt de sistema completo • El prompt de sistema debe indicar a la IA: actuar como socio de pensamiento critico (no resenador), profundizar (no resumir), hacer preguntas que generen mas preguntas, conectar con otras obras cuando sea relevante • Test de integracion: enviar un mensaje simple a Groq y verificar que la respuesta llega correctamente • Manejo de errores de Groq: rate limit (429), timeout, respuesta vacia
LISTO CUANDO	• El backend puede enviar un mensaje a Groq y recibir respuesta sin errores • El prompt de sistema base esta construido y documentado • Los errores de Groq se manejan y retornan mensajes claros al frontend • La API key de Groq nunca aparece en logs ni en respuestas al frontend
ESTUDIAR	• Groq API: documentacion oficial y limites del plan gratuito — console.groq.com/docs • Prompt engineering: como estructurar un system prompt efectivo para un rol especifico • Python: como manejar excepciones de APIs externas con try/except especifico por tipo de error
BLOQUEOS	• *El modelo llama-3.3-70b-versatile en Groq tiene limites de tokens por minuto en capa free. Para el MVP es suficiente, pero hay que conocer los limites para manejar el rate limit correctamente.*

F3-B	**Sesion de analisis con streaming** *Funcionalidades: F-06 completo, F-07*
OBJETIVO	Implementar la sesion de chat completa con streaming visible en tiempo real. El usuario ve las respuestas de la IA aparecer palabra por palabra, no espera a que llegue todo.
TAREAS	• Endpoint POST /analysis/sessions en FastAPI: crea una nueva sesion de analisis para una pelicula. Retorna session_id. • Endpoint POST /analysis/sessions/:id/messages en FastAPI con streaming: recibe el mensaje del usuario, construye el contexto completo (historial de mensajes de la sesion + prompt de sistema), llama a Groq con stream=True, y retorna la respuesta como Server-Sent Events (SSE). • Guardar cada mensaje (usuario y asistente) en la tabla analysis_messages en Supabase al completar el stream • Pagina /analysis/:movieId en el frontend: interfaz de chat con input de texto, boton de enviar, y area de mensajes

	• Implementar el consumo de SSE en el frontend con fetch + ReadableStream: los tokens de la respuesta aparecen en tiempo real • Sugerencias de analisis al inicio de cada sesion: 5 prompts tematicos generados dinamicamente (temas, simbolismo, dilemas eticos, narrativa, contexto historico) que el usuario puede clickear o ignorar • Las sugerencias se generan con una llamada previa a Groq basada en la sinopsis de TMDB de la pelicula • Endpoint PATCH /analysis/sessions/:id/close en FastAPI: marca la sesion como cerrada • Tests: crear sesion, enviar mensaje, verificar que los mensajes se guardan en Supabase
LISTO CUANDO	• El usuario puede iniciar una sesion de analisis desde la pagina de detalle de una pelicula • Las respuestas de la IA aparecen en streaming en tiempo real • Los mensajes se guardan en Supabase correctamente • Las sugerencias de analisis aparecen al inicio de la sesion • La sesion puede cerrarse explicitamente
ESTUDIAR	• Server-Sent Events (SSE): como implementar streaming con FastAPI usando StreamingResponse • fetch API y ReadableStream: como consumir SSE en el frontend sin librerias externas • Groq streaming: como usar stream=True en el SDK de Python y iterar sobre los chunks
BLOQUEOS	• *El streaming puede fallar silenciosamente si hay un error en medio del stream. Implementar manejo de error en el cliente que muestre un mensaje si el stream se corta.* • *Guardar el mensaje completo al finalizar el stream, no durante — si se guarda parcialmente puede corromperse el historial.*

F3-C	**Extraccion de etiquetas semanticas** *Funcionalidades: F-08*
OBJETIVO	Al cerrar una sesion de analisis, el backend extrae automaticamente etiquetas semanticas de la conversacion usando Groq. Estas etiquetas son los datos que construyen el perfil del usuario.
TAREAS	• Funcion extract_semantic_tags(session_id) en el backend: recupera todos los mensajes de la sesion, construye un prompt de extraccion, llama a Groq y pide respuesta en JSON estricto • El prompt de extraccion debe pedir: temas_principales (lista), tipo_narrativa (string), dilemas_eticos (lista), directores_estilo_similar (lista), nivel_filosofico (bajo/medio/alto), palabras_clave (lista) • Validar que la respuesta de Groq es JSON valido antes de guardarla — si no lo es, reintentar una vez • Guardar las etiquetas en la tabla semantic_tags de Supabase vinculadas a la sesion y al usuario • Esta funcion se llama automaticamente cuando el usuario cierra la sesion (endpoint PATCH /analysis/sessions/:id/close)

	• En el frontend, mostrar un indicador de 'Procesando analisis...' mientras se extraen las etiquetas • Mostrar las etiquetas extraidas en la vista de la sesion cerrada • Tests: verificar que la extraccion produce JSON valido, verificar que las etiquetas se guardan en Supabase
LISTO CUANDO	• Al cerrar una sesion, las etiquetas se extraen automaticamente sin accion del usuario • Las etiquetas se guardan en Supabase en formato JSON estructurado • Las etiquetas son visibles en la vista de la sesion cerrada • Si la extraccion falla, la sesion se cierra igualmente — la extraccion no bloquea el flujo principal
ESTUDIAR	• Prompt engineering para extraccion de datos estructurados: como pedir JSON estricto a un LLM de forma confiable • Python json module: como parsear y validar JSON, como manejar JSONDecodeError • FastAPI: como ejecutar una tarea en segundo plano con BackgroundTasks para no bloquear la respuesta al usuario
BLOQUEOS	• *Los LLMs a veces no responden JSON valido aunque se les pida. El reintento y el fallback son obligatorios.*

F3-D	**Historial de conversaciones y conexiones entre peliculas** *Funcionalidades: F-09, F-10*
OBJETIVO	El usuario puede ver y retomar cualquier sesion de analisis pasada. Al iniciar un nuevo analisis, la IA conecta automaticamente con peliculas previas cuando es relevante.
TAREAS	• Endpoint GET /analysis/sessions en FastAPI: retorna todas las sesiones del usuario con metadatos (pelicula, fecha, estado, si tiene etiquetas) • Endpoint GET /analysis/sessions/:id en FastAPI: retorna todos los mensajes de una sesion especifica • Pagina /history en el frontend: lista de todas las sesiones ordenadas por fecha, con poster de la pelicula, titulo, fecha y estado • Al clickear una sesion cerrada: vista de solo lectura con todos los mensajes y las etiquetas extraidas • Al clickear una sesion activa (o al reabrir una cerrada): volver al chat con el historial cargado • Para las conexiones entre peliculas: en la funcion build_analysis_prompt, incluir los titulos y temas principales de las ultimas 5 sesiones cerradas del usuario como contexto adicional • Instruccion en el prompt de sistema: 'Si ves conexiones relevantes con las peliculas anteriores del usuario, mencionalas de forma natural. No fuerces conexiones que no existen.' • Solo activar el contexto de peliculas previas si el usuario tiene al menos 2 sesiones cerradas
LISTO CUANDO	• La pagina /history muestra todas las sesiones del usuario • El usuario puede ver el contenido completo de cualquier sesion pasada

	• El usuario puede reabrir una sesion cerrada y continuar la conversacion • Cuando hay historial suficiente, la IA menciona conexiones con peliculas previas en las nuevas sesiones
ESTUDIAR	• React Router: navegacion entre paginas y como pasar parametros por URL • Como estructurar el contexto de conversacion para Groq: el formato messages con roles user y assistant • Context window de LLMs: por que no se puede incluir todo el historial siempre y como hacer una seleccion inteligente

4 FASE — Perfil del Usuario

El perfil semantico completo — automatico, declarado y personalizable

Al finalizar esta fase, el perfil del usuario esta completamente activo: se construye automaticamente con cada analisis, el usuario puede declarar sus preferencias, escribir instrucciones personalizadas para la IA, y agregar notas de memoria que persisten entre sesiones.

F4-A	**Perfil semantico automatico** *Funcionalidades: F-11*
OBJETIVO	Construir la logica que agrega las etiquetas semanticas de todas las sesiones del usuario en un perfil coherente que crece con cada analisis.
TAREAS	• Funcion build_user_profile(user_id) en el backend: lee todas las etiquetas semanticas del usuario de la tabla semantic_tags, agrega por tipo (temas, directores, narrativa), calcula frecuencias relativas, y retorna un objeto de perfil estructurado • Esta funcion se llama despues de cada extraccion de etiquetas — el perfil se reconstruye completamente cada vez (es mas simple y confiable que actualizaciones incrementales) • Guardar el perfil agregado en la tabla user_profile de Supabase (upsert — un solo registro por usuario) • Endpoint GET /profile/semantic en FastAPI: retorna el perfil semantico actual del usuario • Pagina /profile en el frontend — seccion de perfil semantico: visualizacion de temas frecuentes (lista ordenada por frecuencia), directores afines, tipo de narrativa predominante • Indicador de progreso del perfil: 'Has analizado N peliculas. Analiza X mas para un perfil mas preciso.' • Si el usuario no tiene sesiones cerradas, mostrar estado vacio con llamada a la accion: 'Analiza tu primera pelicula para empezar a construir tu perfil'
LISTO CUANDO	• El perfil semantico se actualiza automaticamente despues de cada sesion cerrada • La pagina /profile muestra los temas frecuentes y directores afines con datos reales • El estado vacio se muestra correctamente cuando no hay sesiones

	• El perfil se incluye como contexto en las nuevas sesiones de analisis
ESTUDIAR	• Python: como agregar listas de diccionarios y calcular frecuencias con Counter • Supabase: como hacer un upsert (insert or update) en una tabla • Como representar visualmente una lista de frecuencias en React sin librerias complejas

F4-B	**Preferencias declaradas, instrucciones personalizadas y memoria** *Funcionalidades: F-12, F-13, F-14*
OBJETIVO	El usuario puede declarar sus preferencias cinematograficas, escribir instrucciones personalizadas para la IA, y agregar notas de memoria. Todo esto enriquece el contexto de cada sesion.
TAREAS	• Endpoint PUT /profile/preferences en FastAPI: recibe y guarda generos favoritos y directores de referencia del usuario • Endpoint PUT /profile/instructions en FastAPI: recibe y guarda el texto libre de instrucciones personalizadas • Endpoint GET /profile/memory y POST/DELETE /profile/memory/:id en FastAPI: CRUD de notas de memoria individuales • En la pagina /profile — seccion Preferencias: selector multiple de generos (lista predefinida de 15 generos comunes), campo de texto para directores de referencia • Seccion Instrucciones personalizadas: textarea con el texto predeterminado cargado, modificable libremente (max 1000 chars), boton de guardar, confirmacion visual de guardado • Texto predeterminado para instrucciones: 'Actua como un interlocutor critico y exigente. No resumas la pelicula. Profundiza en los temas que surjan. Haz preguntas que generen mas preguntas. Cuando sea relevante, conecta con otras obras, ideas o contextos historicos.' • Seccion Memoria: lista de notas activas, boton de agregar nota (modal con textarea, max 200 chars), boton de eliminar nota individual, limite de 10 notas activas • Actualizar la funcion build_analysis_prompt para incluir: preferencias declaradas + instrucciones personalizadas + notas de memoria activas • Tests: guardar preferencias, actualizar instrucciones, agregar y eliminar nota de memoria
LISTO CUANDO	• El usuario puede modificar sus preferencias desde /profile y los cambios se guardan en Supabase • El usuario puede modificar las instrucciones personalizadas y los cambios aplican a la siguiente sesion • El usuario puede agregar hasta 10 notas de memoria • Todas las secciones del perfil (preferencias, instrucciones, memoria) aparecen en el prompt de sistema de las nuevas sesiones • Los tests del CRUD de memoria pasan
ESTUDIAR	• FastAPI: como manejar multiples endpoints en un mismo router con APIRouter

	• React: como manejar un formulario con multiples secciones y guardado independiente por seccion • Como estructurar el prompt de sistema cuando hay multiples fuentes de contexto (perfil semantico + preferencias + instrucciones + memoria) sin que se vuelva incoherente
BLOQUEOS	• *Las instrucciones personalizadas del usuario pueden contradecir el comportamiento base de la IA. El prompt de sistema debe tener un orden de prioridad claro: instrucciones del sistema > instrucciones del usuario > preferencias.*

5 FASE — Recomendaciones

El ciclo de valor se cierra — el perfil genera recomendaciones personalizadas

Al finalizar esta fase, el ciclo central del producto esta completo: el usuario analiza peliculas, construye un perfil, y ese perfil genera recomendaciones que explican exactamente por que conectan con su forma de pensar.

F5-A	**Motor de recomendaciones personalizadas** *Funcionalidades: F-15*
OBJETIVO	Construir el motor que genera recomendaciones de peliculas a partir del perfil semantico del usuario, con razonamiento explicado para cada una.
TAREAS	• Funcion generate_recommendations(user_id, n=5) en el backend: lee el perfil semantico completo del usuario (semantico + preferencias + peliculas ya vistas), construye un prompt de recomendacion, llama a Groq pidiendo N peliculas en JSON con titulo, razon personalizada y temas en comun • El prompt de recomendacion debe incluir: perfil semantico del usuario, lista de peliculas ya vistas (para no repetir), lista de peliculas ya recomendadas (para no repetir), instruccion de generar razonamientos especificos al perfil (no genericos) • Para cada pelicula recomendada, buscar automaticamente el tmdb_id haciendo una busqueda en TMDB API para obtener poster y datos reales • Endpoint GET /recommendations en FastAPI: retorna las recomendaciones actuales del usuario • Endpoint POST /recommendations/generate en FastAPI: triggerea la generacion de nuevas recomendaciones (llamable manualmente desde el frontend) • Endpoint PATCH /recommendations/:id/dismiss en FastAPI: el usuario descarta una recomendacion • Guardar las recomendaciones en la tabla recommendations de Supabase con su razonamiento • Pagina /recommendations en el frontend: grid de peliculas recomendadas con poster, titulo, razon personalizada, temas en comun, y boton 'No me interesa' • Boton 'Generar nuevas recomendaciones' visible cuando el usuario ha descartado todas o quiere mas

	• Si el usuario tiene menos de 3 peliculas analizadas: mostrar estado con mensaje 'Analiza al menos 3 peliculas para recibir recomendaciones precisas' • Tests: generacion produce JSON valido, las peliculas ya vistas no aparecen en las recomendaciones
LISTO CUANDO	• El usuario con 3 o mas peliculas analizadas recibe recomendaciones con razonamiento personalizado • Las recomendaciones incluyen poster y datos reales de TMDB • Las peliculas ya vistas no aparecen nunca en las recomendaciones • El usuario puede descartar recomendaciones y no vuelven a aparecer • El estado vacio con menos de 3 peliculas se muestra correctamente • Los tests pasan
ESTUDIAR	• Prompt engineering para recomendaciones: como pedir al LLM que sea especifico y no generico en sus razonamientos • Como hacer multiples llamadas a TMDB para enriquecer los resultados del LLM sin bloquear el flujo • Python asyncio basico: como hacer llamadas concurrentes a TMDB para las N peliculas recomendadas en paralelo en lugar de secuencial
BLOQUEOS	• *El LLM puede inventar peliculas que no existen. La busqueda en TMDB actua como validacion: si no se encuentra la pelicula, se descarta esa recomendacion y se pide una de reemplazo.* • *Generar recomendaciones consume tokens significativos. Cachear las recomendaciones actuales en Supabase y solo regenerar cuando el usuario lo pide explicitamente.*

6 FASE — Cierre del MVP

Exportacion, pulido, tests end-to-end y deploy estable

Esta fase no agrega funcionalidades nuevas de valor. Completa lo que falta, pule lo que existe, y asegura que el producto esta en condiciones de recibir usuarios reales sin fallar de formas vergonzosas.

F6-A	**Exportacion del historial** *Funcionalidades: F-16*
OBJETIVO	El usuario puede exportar su historial completo de analisis en Markdown o JSON.
TAREAS	• Endpoint GET /export/markdown en FastAPI: genera un archivo .md con todas las sesiones del usuario, cada una con titulo de la pelicula, fecha, mensajes completos y etiquetas extraidas • Endpoint GET /export/json en FastAPI: genera un archivo .json con todos los datos estructurados del usuario (sesiones, mensajes, etiquetas, perfil) • Ambos endpoints retornan el archivo como descarga directa (Content-Disposition: attachment)

	• En el frontend, pagina /profile — seccion Exportar: dos botones, 'Exportar Markdown' y 'Exportar JSON', que triggerian la descarga directamente desde el browser • Indicador de carga mientras se genera el archivo • Si el usuario no tiene sesiones, los botones estan deshabilitados con tooltip explicativo
LISTO CUANDO	• El usuario puede descargar su historial completo en Markdown con un click • El usuario puede descargar su historial completo en JSON con un click • Los archivos descargados contienen datos reales y correctamente formateados • Si no hay sesiones, los botones estan deshabilitados
ESTUDIAR	• FastAPI: como retornar un archivo para descarga con FileResponse o StreamingResponse con headers correctos • Como generar un archivo Markdown bien formateado desde Python con f-strings • Como triggear una descarga de archivo en el browser desde una llamada fetch en JavaScript

F6-B	**Pulido de UI y experiencia general** *Funcionalidades: Ninguna nueva — mejora transversal*
OBJETIVO	Revisar toda la app y corregir inconsistencias visuales, estados de carga faltantes, mensajes de error pobres, y flows que no son intuitivos.
TAREAS	• Revisar todos los estados de carga en la app — cada operacion asincrona debe tener un spinner o skeleton visible • Revisar todos los mensajes de error — deben ser en espanol, claros para el usuario, y nunca mostrar stacktraces o mensajes tecnicos • Responsive: verificar que todas las paginas se ven bien en mobile (375px) y desktop (1280px) minimo • Modo oscuro: implementar toggle de modo oscuro con Tailwind dark: classes y persistencia en localStorage • Navegacion: verificar que el flujo entre todas las paginas es logico y no hay callejones sin salida • Empty states: todas las listas vacias deben tener un mensaje util con llamada a la accion, no solo estar en blanco • Confirmaciones: todas las acciones destructivas (eliminar pelicula, eliminar sesion, eliminar nota) tienen confirmacion • Accesibilidad basica: todos los botones tienen aria-label, los inputs tienen labels, el contraste de colores es legible
LISTO CUANDO	• Todas las operaciones asincronas tienen estado de carga visible • Todos los mensajes de error son en espanol y utiles para el usuario • La app se ve bien y es usable en mobile • El modo oscuro funciona y persiste entre sesiones • No hay listas vacias sin estado vacio definido
ESTUDIAR	• Tailwind CSS dark mode: como configurar y usar dark: classes — tailwindcss.com/docs/dark-mode

	• Web accessibility basica: que es aria-label y cuando usarlo — developer.mozilla.org/accessibility

F6-C	**Tests end-to-end y verificacion del ciclo completo** *Funcionalidades: Ninguna nueva — verificacion*
OBJETIVO	Verificar que el ciclo central del producto funciona de principio a fin sin errores, y que los casos de error mas comunes estan cubiertos.
TAREAS	• Ejecutar el ciclo completo manualmente: registrar usuario > buscar pelicula > marcarla como vista > iniciar analisis > conversar > cerrar sesion > verificar etiquetas > verificar perfil actualizado > generar recomendaciones • Verificar que RLS de Supabase funciona: crear dos usuarios, verificar que usuario A no puede ver datos de usuario B bajo ningun flujo • Tests de integracion backend: al menos un test por endpoint critico (register, login, create session, send message, close session, get profile, get recommendations) • Verificar comportamiento con Groq en rate limit: que el mensaje de error al usuario es util y no rompe la app • Verificar comportamiento con TMDB sin resultados: busqueda de string sin resultados reales • Verificar que ninguna API key aparece en los logs de Render o Vercel • Verificar que el deploy en produccion (no local) funciona correctamente para todos los flujos
LISTO CUANDO	• El ciclo completo funciona de principio a fin en el deploy de produccion • RLS esta verificado — usuarios aislados correctamente • Todos los tests de integracion criticos pasan • Los casos de error de Groq y TMDB muestran mensajes utiles al usuario • Ninguna API key aparece en logs
ESTUDIAR	• FastAPI TestClient: como escribir tests de integracion para endpoints — fastapi.tiangolo.com/tutorial/testing • Como revisar los logs de Render y Vercel para detectar errores en produccion
BLOQUEOS	• *Los errores que solo aparecen en produccion (variables de entorno faltantes, CORS, URLs hardcodeadas) son los mas dificiles de debuggear. Probar en produccion desde F0-C minimiza esta sorpresa.*

Reglas de trabajo

Regla	Descripcion
Una subfase a la vez	No se inicia la siguiente subfase hasta que todos los criterios de la actual se cumplen. Sin excepciones.

Deploy continuo desde F0-C	Cada subfase terminada se mergea a main y se verifica en produccion, no solo en local.
TDD en logica de negocio	Las funciones de extraccion de etiquetas, construccion de perfil, y generacion de recomendaciones tienen tests antes de implementacion.
TypeScript strict en todo	Sin any ni @ts-ignore. Si TypeScript se queja, el codigo se arregla, no se silencia.
Python type hints en todo	Todas las funciones del backend tienen tipos anotados en parametros y retorno.
Variables de entorno siempre	Ninguna API key, URL de Supabase, ni credencial aparece en el codigo fuente bajo ninguna circunstancia.
Commits descriptivos en ingles	El historial de Git es parte del portfolio. Cada commit debe describir exactamente que cambia.
Estudiar segun necesidad	La seccion ESTUDIAR de cada subfase es el curriculo. Se estudia lo que se necesita para construir, no antes ni despues.

Siguiente paso

El Roadmap esta completo. El siguiente paso es empezar la Fase 0. Antes de escribir una sola linea de codigo de producto, F0-A, F0-B y F0-C deben estar completamente terminadas.

EMPIEZA POR	F0-A — Estructura del monorepo. Crear el repositorio en GitHub y la estructura de carpetas del proyecto. Sin eso, nada mas puede construirse.